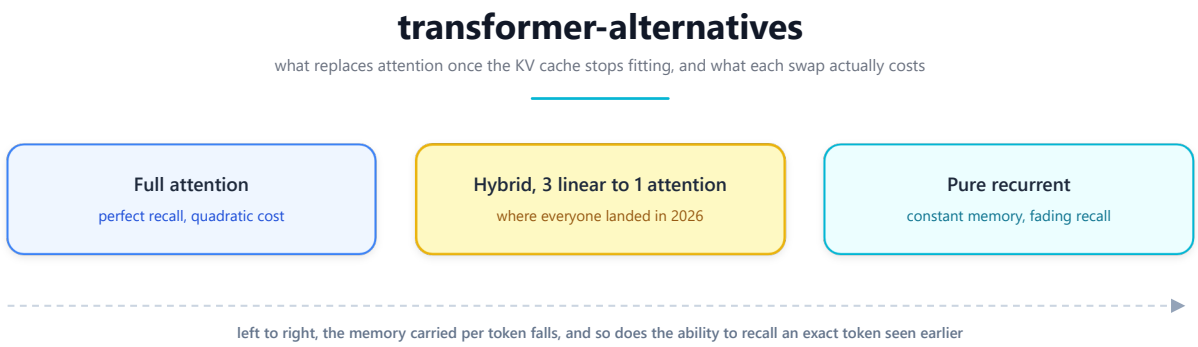
🌊 transformer-alternatives

families 4 type learning notes

license MIT contributions welcome



📝 Project Description

Attention is not being replaced because it works badly. It is being replaced because **it remembers by keeping everything**. Every token generated adds a row to a KV cache that never shrinks, so memory grows with context and compute grows with its square.

Every alternative here makes the same trade. **Replace a cache that grows with a state that does not**, and accept that a fixed state eventually forgets. These learning notes are the third of a series, after Original LLM (the 2017 Transformer) and Modern LLM (everything that changed **inside** it).

📊 The bill and the price

Context	Attention, plain MHA	Attention, GQA at 8 kv heads	Mamba-2 state
1 k tokens	0.5 GB	0.13 GB	0.13 GB
8 k tokens	4 GB	1 GB	0.13 GB
32 k tokens	16 GB	4 GB	0.13 GB
128 k tokens	64 GB	16 GB	0.13 GB

The bill, for a 7B model with 32 layers. Read the last column, it never moves. **GQA divides the bill, a recurrent state flattens it.**

```
prompt   "... alice -> 42 ...  bob -> 17 ...  carol -> 99 ...   what is bob?"

full attention          looks the position up directly, correct at any depth
fixed state, 64 slots   correct until the 64th pair, then it starts guessing
hybrid, 1 attention     correct, that single layer does the lookup for the others
```

The price, measured by Multi-Query Associative Recall (MQAR). A fixed state does not degrade gracefully, it fails **once it is full**.

▼ 📐 Why a constant factor is not enough

GQA is a constant factor, a recurrent state is a change of exponent, and that difference is the entire argument. On a 6 GB card the practical consequence is blunt: at 32 k tokens the KV cache alone leaves no room for the weights it belongs to.

▼ 🧱 Why a full state cannot be trained away

Once the state is full, everything written after overwrites something written before, and no amount of training fixes a capacity limit. This is why pure recurrent models look excellent on perplexity and disappoint on "find the one line in this file".

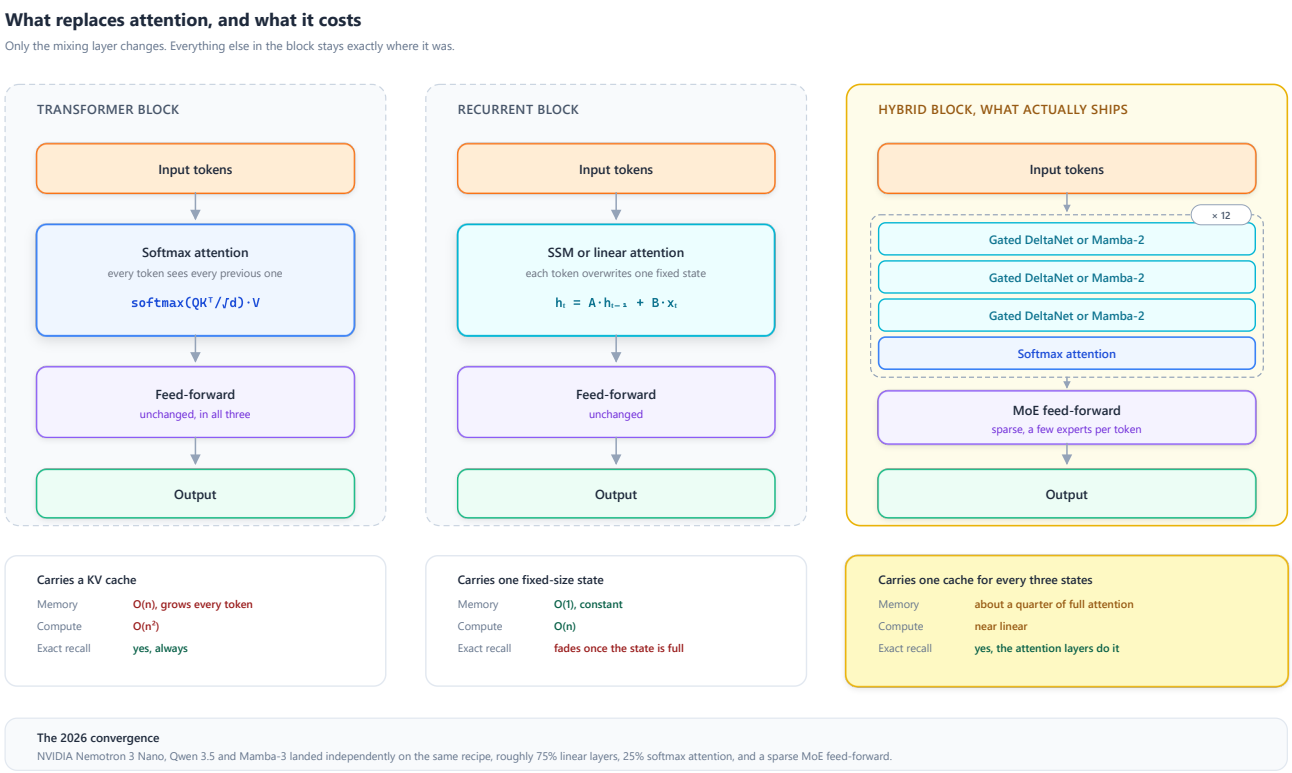
▼ 📈 Why perplexity hides the problem

Averaged over a corpus, forgetting one rare token costs almost nothing. Zoology pinned the number down, **82% of the perplexity gap** between attention-free models and attention comes from recall alone, which is why MQAR became the standard probe.

▼ ⚖️ Nobody ships pure

Every serious 2026 release keeps some softmax attention. The question stopped being *whether* to keep it and became *how little* is enough.

How it works

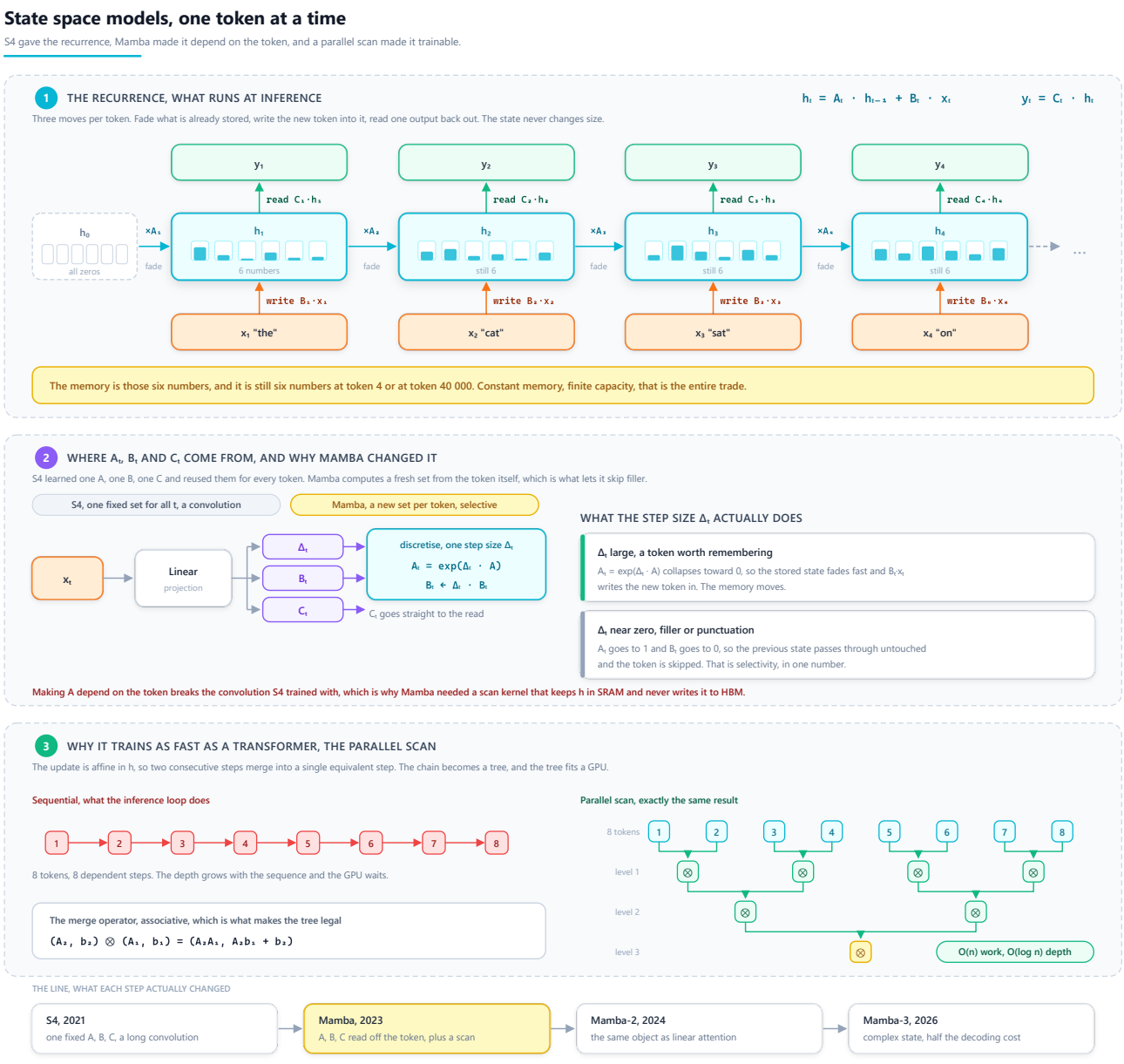


Three blocks, one difference. The left keeps every token, the middle keeps one state, the right keeps one cache for every three states. The four families below fill in that middle and right column.

---

1 State space models

A recurrence borrowed from control theory, made trainable in parallel.



Selectivity is the whole idea. A fixed `A` filters every token the same way, which is a convolution wearing a costume. Making it input-dependent is what turned a nice long-range model into a language model.

▼ 📄 Code · the recurrence and the parallel scan

```
# What runs at inference, one token at a time. h is the entire memory.
for t in range(seq_len):
    h = A[t] * h + B[t] * x[t]
    y[t] = C[t] @ h

# The same maths at training time. This is the breakthrough, not the recurrence.
y = associative_scan(A, B, C, x)   # O(n) work, O(log n) depth on a GPU
```

▼ 🕰️ **From S4 to Mamba-3**

S4 came from signal processing, not from language. A structured state matrix plus a discretisation step turns a continuous system into something that handles 16 k tokens without attention.

Mamba made `A`, `B` and `C` depend on the input. That selectivity is what lets it skip filler and keep what matters. It broke the convolutional shortcut, so the answer was a hardware-aware scan that never writes the state to HBM.

Mamba-2 proved SSMs and linear attention are the same object (state space duality), which unlocked plain matmul kernels. **Mamba-3** (ICLR 2026) adds a complex-valued state update for richer state tracking and a MIMO form, matching strong baselines at **half the decoding cost**.

---

2 Linear attention and modern RNNs

Drop the softmax and the algebra reassociates, which is all it takes.

Linear attention and modern RNNs, one bracket move

Take the softmax out and the same product can be grouped the other way. The cache becomes one fixed matrix, and every paper since is about how to forget it.

1 THE BRACKET MOVE, AND WHY IT DELETES THE CACHE

The softmax sits between $q \cdot K^T$ and V, so the product has to be evaluated left to right. Remove it and the right pair can go first, which changes what has to be stored.

SOFTMAX ATTENTION — $O(n^2)$ time

$$y = (q \cdot K^T) \cdot V$$

the softmax forces that grouping, so the first result has shape

`1 × n one score for every past token`

so every past k and v has to be kept, one row per token

k_1 , v_1
k_2 , v_2
k_3 , v_3 …

one row added per token, and it is never freed

LINEAR ATTENTION — $O(n)$ time

$$y = \phi(q) \cdot (\phi(K)^T \cdot V)$$

nothing in the way now, so the right pair goes first and gives

`d × d`

n has left the shape entirely. Call it S. It is the same matrix at 1 k tokens and at 1 M tokens.

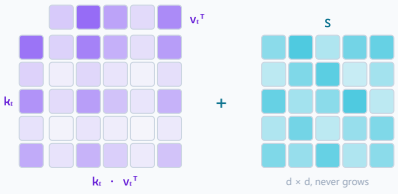
S is the whole memory, and it is updated in place

$S \leftarrow S + k_t \cdot v_t^T$ one add per token, nothing kept per token

2 WHAT THAT MATRIX ACTUALLY IS, A WRITE AND A READ

Each token writes one outer product into S. The query pulls a mixture back out. This is an RNN, and it is exactly what runs at inference time.

WRITE, token t adds one rank-one update



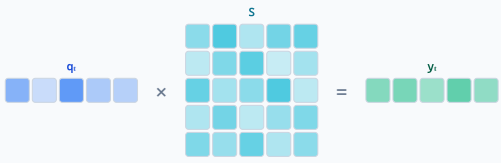
Rank one

every row of the grid is the same v_t pattern, scaled by that row's k_t.

Nothing about token t is stored, it is folded in.

$S_t = S_{t-1} + k_t \cdot v_t^T$

READ, the query pulls a mixture back out



$y_t = q_t \cdot S$ every pair ever written contributes, weighted by how much its key resembles q_t

S is an associative memory. Write a key with its value, query it back later. When two keys look alike their values blend, and no fixed state can undo that blend.

3 EVERY PAPER SINCE IS ABOUT HOW TO FORGET

A plain running sum saturates, so old pairs blur into new ones. Each line below changes the middle term of the update, and nothing else.

ARCHITECTURE	THE UPDATE RULE, THE ONLY THING THAT MOVES	WHAT IT BUYS
Linear attention, 2020	$S_t = S_{t-1} + k_t v_t^T$	nothing. It adds forever, so the matrix saturates and old pairs blur.
RetNet, 2023	$S_t = \gamma \cdot S_{t-1} + k_t v_t^T$	one fixed decay γ, so the past fades on a schedule. Same for every token.
GLA, 2024	$S_t = \mathrm{Diag}(\alpha_t) \cdot S_{t-1} + k_t v_t^T$	a gate read off the token, so it forgets channel by channel, on demand.
DeltaNet, 2024	$S_t = S_{t-1}(I - \beta_t k_t k_t^T) + \beta_t k_t v_t^T$	erases whatever was stored at that key, then writes. A replace, not an add.
Gated DeltaNet, 2025	`the gate and the replace, together`	forgets on demand and overwrites cleanly. What the 2026 hybrids ship.

Read down the middle column. The write, the read and the shape of S are identical in all five, only the term in front of S_{t-1} ever changes.

TWO ROADS, ONE DESTINATION

Coming from attention
drop the softmax, reassociate, then bolt a decay, a gate or a delta rule back on to control the forgetting

→ the same recurrence ←

Coming from RNNs
RWKV-7 generalises the delta rule, xLSTM adds multiplicative gating, and xLSTM-7B trains 3.5x faster than an equal transformer

Two roads, one destination. One camp removed the softmax from attention, the other fixed the RNN so it trains in parallel. They met in the middle, and the modern papers are barely distinguishable.

▼ The bracket move, in two lines

```
attention          softmax(q · kᵀ) · v          the softmax blocks reassociation, so O(n²)

linear attention   (ϕ(q) · ϕ(k)ᵀ) · v
                 = ϕ(q) · (ϕ(k)ᵀ · v)           one fixed d × d matrix S, carried forward
                            └─ a running sum, updated per token
```

`kᵀv` becomes a running sum instead of a matrix to rebuild, so the model is an RNN at inference and a parallel scan at training, with no cache in sight.

▼ 🚪 How each model forgets

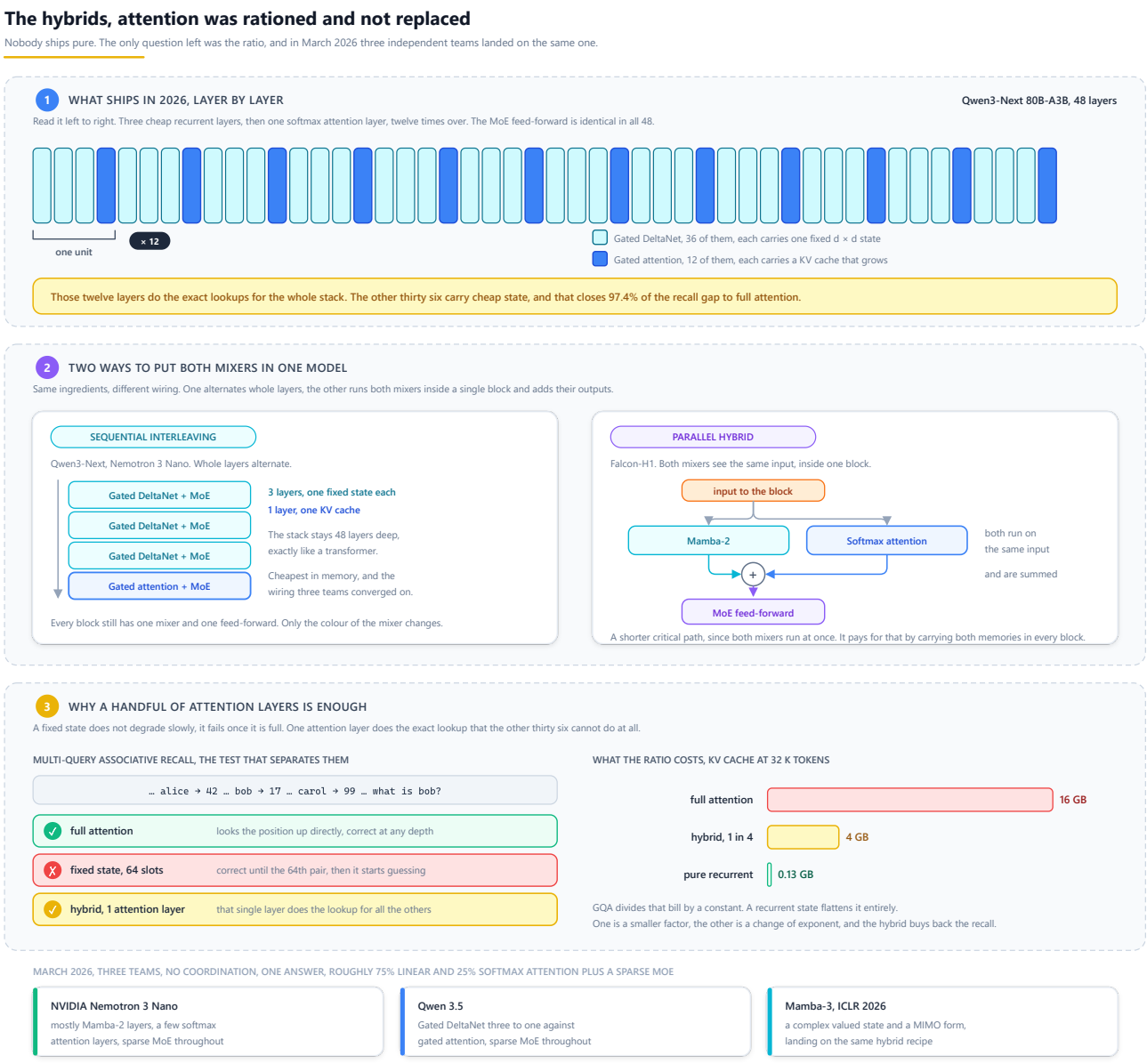
A plain sum saturates, so everything since is about forgetting. RetNet added a fixed exponential decay, GLA a data-dependent gate, and DeltaNet a write rule that **replaces** an association instead of adding to it. Gated DeltaNet combines both and is what most 2026 hybrids actually use.

▼ 🐓 RWKV and xLSTM, from the RNN side

They arrive from the other camp and land in the same place. RWKV-7 generalises the delta rule, and xLSTM-7B trains **3.5x faster** than a transformer of equal size through multiplicative gating.

3 The hybrids

Nobody ships pure. The interesting question is the ratio.



> **Attention was not replaced, it was rationed.** The 2026 architecture is a transformer that pays for full attention only where full attention actually earns its cost.

▼ Qwen3-Next, layer by layer

```
Qwen3-Next 80B-A3B, 48 layers

  ┌ Gated DeltaNet ┐
  │ Gated DeltaNet │  × 12   →   36 linear layers, 12 attention layers
  │ Gated DeltaNet │             MoE feed-forward throughout
  └ Gated attention┘
```

Those twelve attention layers do the exact lookups, the other thirty-six carry cheap state. Zoology measured hybrids closing **97.4% of the gap** to full attention while keeping sub-quadratic scaling, and cache memory drops to roughly a quarter.

🔀 Two ways to mix

Sequential interleaving (Qwen3-Next, Nemotron) alternates layers. Parallel hybrids (Falcon-H1) run Mamba-2 and attention **side by side in the same block** and sum them, which trades memory for a shorter critical path.

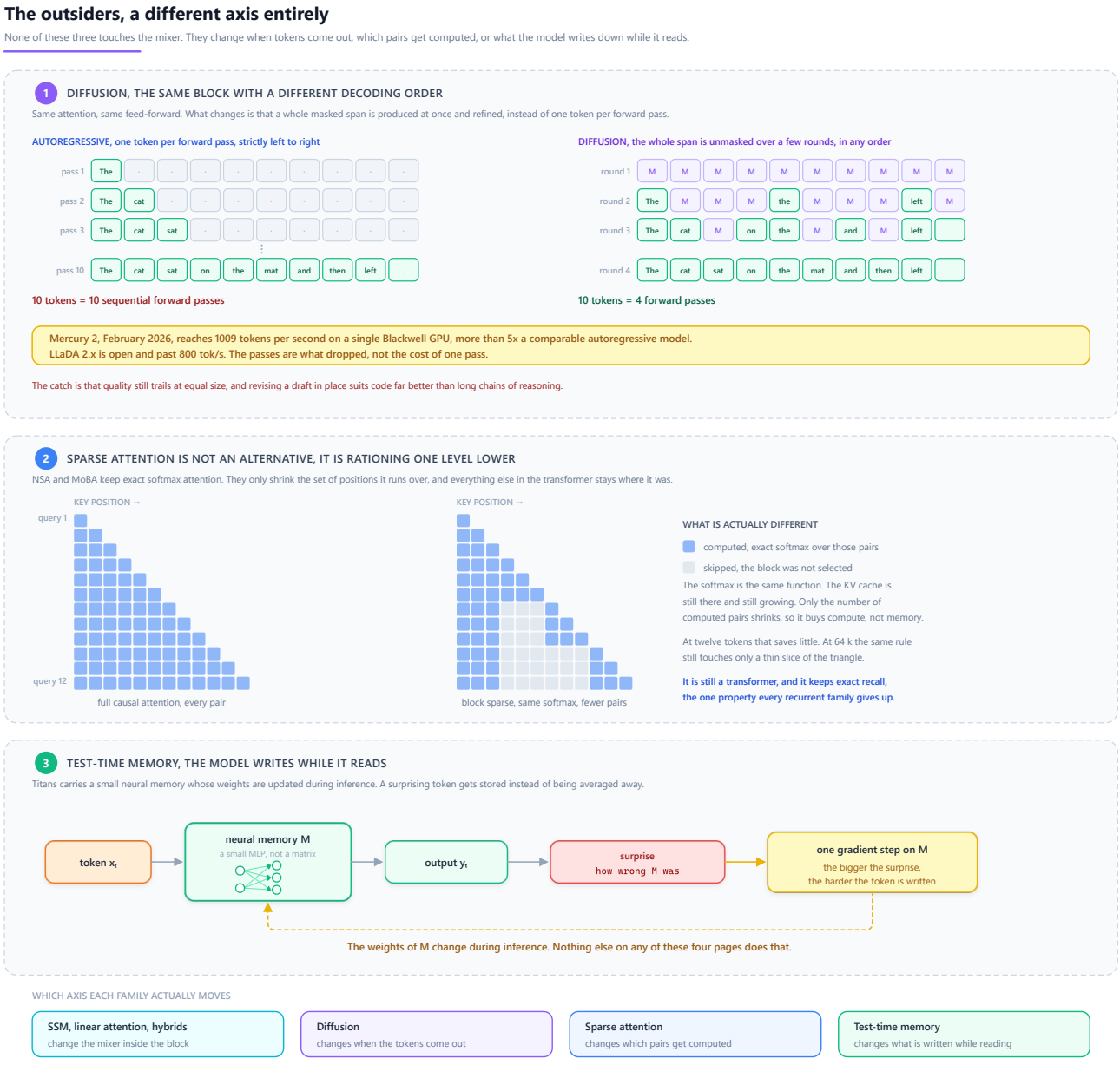
🎯 How March 2026 settled it

NVIDIA Nemotron 3 Nano, Qwen 3.5 and Mamba-3 landed independently on the same recipe, roughly **75% linear, 25% attention, plus a sparse MoE**. Three teams, no coordination, one answer.

---

4 The outsiders

Different axis entirely, these change when tokens are produced, not how they mix.



▼ Diffusion language models

```
autoregressive   one token per forward pass, left to right     n passes for n tokens
diffusion        a whole masked passage, refined in k rounds   k passes, k far below n
```

They shipped. Mercury 2 (February 2026) reaches **1009 tokens per second** on a single Blackwell GPU, over 5x comparable autoregressive models, and LLaDA 2.x is open and past 800 tok/s. The catch is that quality still trails at equal size, and editing a draft in place fits code better than long reasoning.

<details>
<summary>🕳️ Why sparse attention is not an alternative</summary>

NSA and MoBA keep exact softmax attention and simply compute it over selected blocks. It is rationing again, one level lower, and it stays inside the transformer.
</details>

<details>
<summary>🧠 Test-time memory</summary>

Titans keeps a small neural memory updated while it reads, so surprising tokens get stored rather than averaged away. Closer to giving the model a notebook than to changing its mixer.
</details>

🧠 What actually decides, in five lines

1. **The cache is the problem**, not the attention. Every alternative is an answer to a memory bill.
2. **A fixed state is a capacity, not a compression.** It does not degrade slowly, it fails once full.
3. **Parallel training was the unlock.** A recurrence nobody can train in parallel is a research curiosity.
4. **Perplexity will not show you the difference.** Retrieval will, which is why MQAR exists.
5. **The answer was a ratio.** Keep the attention you need, replace the rest, and that number turned out to be about one in four.

<details>
<summary>📚 My other learning notes</summary>

- llm-harness (the agent loop that runs a trained model)
</details>

📖 Inspiration / Sources

This project is a study, so the sources matter more than usual:

- Mamba, Linear-Time Sequence Modeling with Selective State Spaces (Gu and Dao, 2023)
- Transformers are SSMs, state space duality (Dao and Gu, 2024), the paper that unified the two camps
- Mamba-3, Improved Sequence Modeling using State Space Principles (ICLR 2026)
- Simple linear attention language models balance the recall-throughput tradeoff (Arora et al., 2024), where MQAR comes from
- xLSTM, Extended Long Short-Term Memory (Beck et al., 2024)
- RWKV, Reinventing RNNs for the Transformer Era (Peng et al., 2023)
- Zoology, Measuring and Improving Recall in Efficient Language Models (Arora et al., 2023), the analysis that made the recall gap measurable
- Original LLM (where the series starts, a bigram model and a 2017 Transformer)

- Modern LLM (my own model, everything that changed inside the block)

Notes written by me 😎, Thibault GAREL - Github